advance the maximum number of items that will be stored in a Bloom filter at the same time, we have an exact formula to compute the total amount of memory needed to achieve an arbitrarily low ratio of false positives.

---

[1] *Scale up* means moving your application to a more powerful, more expensive machine, while with *scale out* we usually refer to re-designing an application to run in a distributed architecture over several cheaper machines. While there is an upper limit to the possibility of scaling up (the most powerful machine that can be bought by your company – and price grows exponentially in the higher end), with proper design it is possible – in theory – to scale out indefinitely.

[2] We used the term associative array here to avoid confusion between the *dictionary problem* and the *dictionary* abstract data type. Though the two terms are connected, they are not the same thing.

[3] This reasoning is also the basis for the TF-IDF metric used in text search and text analysis: short for **term frequency–inverse document frequency**, TF-IDF is computed as the ratio of raw occurrences of a term in a document (TF), over the logarithm of another fraction, the total number of documents in a *corpus* divided by the number of *corpus* documents in which the term appears (IDF). This means that TF-IDF will be large when a term appears often in a document but rarely in the *corpus*, and small when, conversely, the term is used frequently in many documents.

[4] Binary search takes its name from the fact that you always search the middle of a list, splitting it in two parts: one before the search location, and one after it. Depending on how the element you are looking for compares to the one in the search location, you then might recursively check the first half or the second half of your list (or neither, if you just found what you were looking for).

[5] Not to be confused with the index generated by the hashing function.

[6] At least not from their basic version, but we'll see that some variants have been developed to also handle elements removal.

[7] Double hashing is a technique used to resolve hash collisions (see appendix C, sections C.2.2-C.2.3); when a collision occurs, it adds an offset to the initial position computed, by using a secondary hash of the key. Triple hashing, instead, computes the offset by using a linear combination of 2 auxiliary hash functions.

[8] Dillinger, Peter C., and Panagiotis Manolios. "Fast and accurate bitstate verification for SPIN." International SPIN Workshop on Model Checking of Software. Springer, Berlin, Heidelberg, 2004.